

```

#include <Servo.h>
#include <time.h>
#include <stdlib.h>
#include <pico/time.h>

/* Pin number defines */
#define SERVO_PIN 22
#define SERVO_ANALOG 26
#define TIME_ANALOG 28
#define OFFSET_SELECT_PIN
#define ANGLE_SELECT_PIN 13
#define TIME_SELECT_PIN 19
#define DATA_PIN 18
#define LATCH_PIN 20
#define CLOCK_PIN 21
const int DIGIT_PIN[] = {17, 16, 15, 14};

/* Servo arm angle restrictions */
#define USER_MINIMUM_ANGLE 75
#define USER_MAXIMUM_ANGLE 125

/* Prototypes */
void selectDigit(uint8_t value);
void writeData(int value);
void setDigits(int value);
void writeDisplay(int value, int a[]);

/* Servo control values */
Servo mainServo;
int pos = 0;
int setAngle = USER_MINIMUM_ANGLE;

/* Time control values */
int timer = 0;
int countdown = 0;
int seconds = 0;
int offset = 0;
uint32_t now = to_ms_since_boot(get_absolute_time());
uint32_t stop = now + (seconds * 1000);

/* Display control values */
int digitValue;
int digits[4];

/* Loop control values */
bool startToggle = false;
bool timeToggle = false;
bool angleToggle = false;
bool offsetToggle = true;

/* Values that will draw a circle around the display */
struct circle {
    int digit;
    uint8_t shape;
} sections[12] = {
    {.digit = 3, .shape = 0x02},
    {.digit = 3, .shape = 0x04},
    {.digit = 3, .shape = 0x08},
    {.digit = 2, .shape = 0x08},
    {.digit = 1, .shape = 0x08},

```

```

    {.digit = 0, .shape = 0x08},
    {.digit = 0, .shape = 0x10},
    {.digit = 0, .shape = 0x20},
    {.digit = 0, .shape = 0x01},
    {.digit = 1, .shape = 0x01},
    {.digit = 2, .shape = 0x01},
    {.digit = 3, .shape = 0x01}
};

/* Hex values representing characters 0 through 9, then A through F */
uint8_t num[] = {0x3f, 0x06, 0x5b, 0x4f, 0x66, 0x6d, 0x7d, 0x07,
                 0x7f, 0x6f, 0x77, 0x7c, 0x39, 0x5e, 0x79, 0x71};

/* Pin initialization */
void setup() {
    mainServo.attach(SERVO_PIN, 500, 2500);
    pinMode(ANGLE_SELECT_PIN, INPUT_PULLUP);
    pinMode(TIME_SELECT_PIN, INPUT_PULLUP);
    pinMode(LATCH_PIN, OUTPUT);
    pinMode(CLOCK_PIN, OUTPUT);
    pinMode(DATA_PIN, OUTPUT);
    for (int i = 0; i < 4; i++) {
        pinMode(DIGIT_PIN[i], OUTPUT);
    }
    setAngle = map(analogRead(SERVO_ANALOG), 0, 1023, USER_MAXIMUM_ANGLE, USER_MIN
IMUM_ANGLE);
    seconds = map(analogRead(TIME_ANALOG), 0, 1023, 60, 0);
}

void loop() {
    while (!timeToggle && !angleToggle) {
        for (int i = 0, pos = setAngle; pos < setAngle + 6; pos += 1, i++) {
            mainServo.write(pos);
            for (int j = 0; j < 4; j++) {
                selectDigit(sections[i].digit);
                writeData(sections[i].shape);
                delay(5);
            }
        }
        for (int i = 6, pos = setAngle; pos > setAngle - 6; pos -= 1, i++) {
            mainServo.write(pos);
            for (int j = 0; j < 4; j++) {
                selectDigit(sections[i].digit);
                writeData(sections[i].shape);
                delay(5);
            }
        }
        if (offsetToggle == true) {
            srand((unsigned) time(NULL));
            offset = (rand() % 2000);
        }
        else
            offset = 0;
        now = to_ms_since_boot(get_absolute_time());
        timer = (now + (seconds * 1000) + offset);

        while (to_ms_since_boot(get_absolute_time()) < (now + (seconds * 1000) + off
set)) {
            countdown = timer - to_ms_since_boot(get_absolute_time());
            digits[0] = ((countdown % 1000000) / 100000);

```

```

digits[1] = ((countdown % 100000) / 10000);
digits[2] = ((countdown % 10000) / 1000);
digits[3] = ((countdown % 1000) / 100);
for (int i = 0; i < 4; i++) {
    selectDigit(i);
    digitValue = digits[i];
    if (i == 2) {
        writeData(num[digitValue] | 0x80);
    }
    else {
        writeData(num[digitValue]);
    }
    delay(5);
    writeData(0x00);
}
for (int i = 0; i < 4; i++) {
    selectDigit(i);
    writeData(0x00);
}
if (digitalRead(TIME_SELECT_PIN) == LOW) {
    delay(20);
    if (digitalRead(TIME_SELECT_PIN) == LOW) {
        timeToggle = true;
        startToggle = false;
        break;
    }
}
if (digitalRead(ANGLE_SELECT_PIN) == LOW) {
    delay(20);
    if (digitalRead(ANGLE_SELECT_PIN) == LOW) {
        angleToggle = true;
        startToggle = false;
        break;
    }
}
}
}
}
delay(200);

while (timeToggle) {
    seconds = map(analogRead(TIME_ANALOG), 0, 1023, 60, 0);
    digits[1] = ((seconds % 1000) / 100);
    digits[2] = ((seconds % 100) / 10);
    digits[3] = (seconds % 10);
    for (int i = 1; i < 4; i++) {
        selectDigit(i);
        digitValue = digits[i];
        writeData(num[digitValue]);
        delay(5);
        writeData(0x00);
    }
    if (digitalRead(TIME_SELECT_PIN) == LOW) {
        delay(20);
        if (digitalRead(TIME_SELECT_PIN) == LOW)
            timeToggle = false;
    }
}

while (angleToggle) {
    setAngle = map(analogRead(SERVO_ANALOG), 0, 1023, USER_MAXIMUM_ANGLE, USER_M

```

```

INIMUM_ANGLE);
    mainServo.write(setAngle);
    digits[1] = ((setAngle % 1000) / 100);
    digits[2] = ((setAngle % 100) / 10);
    digits[3] = (setAngle % 10);
    for (int i = 1; i < 4; i++) {
        selectDigit(i);
        digitValue = digits[i];
        writeData(num[digitValue]);
        delay(5);
        writeData(0x00);
    }
    if (digitalRead(ANGLE_SELECT_PIN) == LOW) {
        delay(20);
        if (digitalRead(ANGLE_SELECT_PIN) == LOW)
            angleToggle = false;
    }
}

void selectDigit(uint8_t value) {
    for (int i = 0; i < 4; i++) {
        digitalWrite(DIGIT_PIN[i], HIGH); // Clear all 7-segment sections
    }
    digitalWrite(DIGIT_PIN[value], LOW); // Open the selected individual 7-segment
    t display
}

void writeData(int value) {
    digitalWrite(LATCH_PIN, LOW);
    shiftOut(DATA_PIN, CLOCK_PIN, LSBFIRST, value); // Send serial data to 74HC59
5
    digitalWrite(LATCH_PIN, HIGH); // High level will update data to parallel output
    t (74HC595)
}

```